
MODULE 4

Strings

4.1 Introduction

- ✓ “A string is a sequence of characters enclosed within double quotes”. or
- ✓ “String is an array of characters and terminated by NULL character which is denoted by ‘\0’.
- ✓ In C, a string is a null-terminated character array.
- ✓ This means that after the last character, a null character (‘\0’) is stored to signify the end of the character array.

For example, if we write `char str[] = "HELLO";`

we are declaring an array that has five characters, namely, H, E, L, L, and O. Apart from these characters, a null character (‘\0’) is stored at the end of the string. So, the internal representation of the string becomes HELLO‘\0’. To store a string of length 5, we need 5 + 1 locations (1 extra for the null character). The name of the character array (or the string) is a pointer to the beginning of the string.

- ✓ Figure 4.1 shows the difference between character storage and string storage.

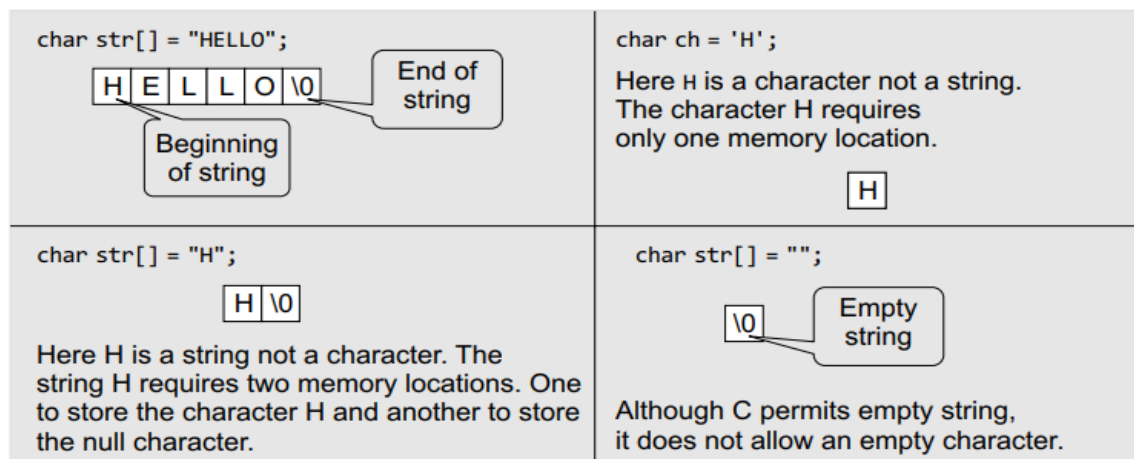


Figure 4.1 Difference between character storage and string storage

- ✓ If we had declared str as
`char str[5] = "HELLO";`

then the null character will not be appended automatically to the character array. This is because str can hold only 5 characters and the characters in HELLO have already filled the space allocated to it.

- ✓ Like we use **subscripts** (also known as **index**) to **access the elements of an array**, we can also use **subscripts to access the elements of a string**. The subscript **starts with a zero (0)**. All the characters of a string are stored in successive memory locations. Figure 4.2 shows how str[] is stored in the memory.

str[0]	1000	H
str[1]	1001	E
str[2]	1002	L
str[3]	1003	L
str[4]	1004	O
str[5]	1005	\0

Figure 4.2 Memory representation of a character array

-
- ✓ ASCII code of a character is stored in the memory and not the character itself. So, at address 1000, 72 will be stored as the ASCII code for H is 72. The statement `char str[] = "HELLO";`

Syntax:

the general form of declaring a string is

`char str[size];`

- ✓ The other way to initialize a string is to initialize it as an **array of characters**. For example,
`char str[] = {'H', 'E', 'L', 'L', 'O', '\0'};`
Here, the compiler will automatically calculate the size based on the number of characters.
- ✓ We can also declare a string with **size much larger than the number of elements** that are initialized. For example, consider the statement below.
`char str [10] = "HELLO";`
In such cases, the compiler creates an array of size 10; stores "HELLO" in it and finally terminates the string with a null character. Rest of the elements in the array are automatically initialized to NULL
- ✓ Now consider the following statements:
`char str[3];`
`str = "HELLO";`
The above initialization statement is **illegal in C and would generate a compile-time error**.

4.1.1 Reading Strings

- ✓ If we declare a string by writing `char str[100];` Then str can be read by the user in three ways:
 1. using **scanf function**,
 2. using **gets() function**, and
 3. using **getchar(), getch() or getche()** function repeatedly.

using scanf()

- ✓ Strings can be read using `scanf()` by writing **`scanf("%s", str);`**
- ✓ Unlike int, float, and char values, %s format does not require the ampersand before the variable str.
- ✓ The main **pitfall** of using this function is that the **function terminates as soon as it finds a blank space. Therefore we cannot read the complete sentence using scanf() function.**

using gets()

- ✓ The string can be read by writing **`gets(str);`**
- ✓ `gets()` is a simple function that **overcomes the drawbacks of the scanf() function.**
- ✓ **`gets() function is used to read a sequence of characters (string) with spaces in between.`**
- ✓ The 'gets()' function allows us to read an 'entire line' of input including whitespace characters.
- ✓ The `gets()` function takes the starting address of the string which will hold the input.
- ✓ The string inputted using `gets()` is **automatically terminated with a null character.**

using getchar()

- ✓ Strings can also be read by calling the **`getchar() function repeatedly to read a sequence of single characters`** (unless a terminating character is entered) and simultaneously storing it in a character array as shown below:

```
i=0;
ch = getchar;// Get a character
while(ch != '\0')
{
    str[i] = ch;// Store the read character in str
    i++;
    ch = getchar();// Get another character
}
str[i] = '\0';// Terminate str with null character
```

4.1.2 Writing Strings

- ✓ Strings can be displayed on the screen using the following three ways:
 1. using **printf() function**
 2. using **puts() function**, and
 3. using **putchar() function** repeatedly.

using printf()

- ✓ Strings can be displayed using printf() by writing **printf("%s", str);**
- ✓ We use the **format specifier %s** to output a string. Observe carefully that there is **no '&' character** used with the string variable.
- ✓ We may also use **width and precision specifications** along with %s.
- ✓ The precision specifies the maximum number of characters to be displayed, after which the string is truncated. For example, printf ("%5.3s", str); The above statement would print only the first three characters in a total field of five characters. Also these characters would be right justified in the allocated width.
- ✓ To make the string left justified, we must use a minus sign. For example, printf ("%–5.3s", str);

using puts()

- ✓ A string can be displayed by writing **puts(str);**
- ✓ puts() is a simple function that overcomes the drawbacks of the printf() function.
- ✓ The puts() function **writes a line of output on the screen**. It terminates the line with a newline character ('\n').

using putchar()

- ✓ Strings can also be written by calling the **putchar() function repeatedly to print a sequence of single characters**.

```
i=0;
while(str[i] != '\0')
{
    putchar(str[i]); // Print the character on the screen
    i++;
}
```

4.2 String Taxonomy

- ✓ In C, we can store a string either in **fixed-length format or in variable-length format** as shown in Figure 13.4.

4.2.1 Fixed-length strings

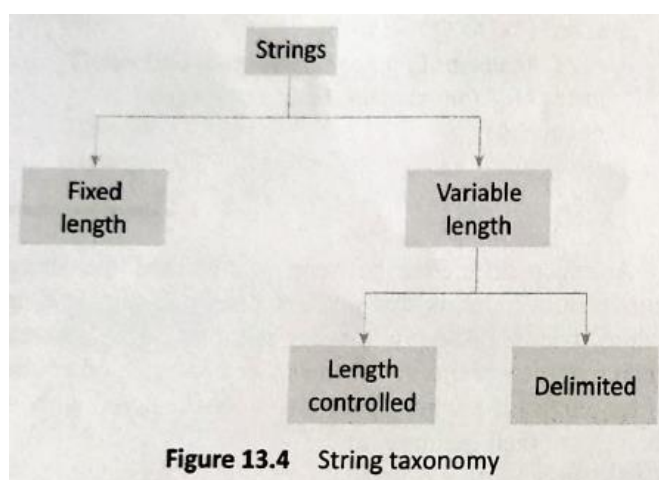
- ✓ When storing a string in a fixed-length format, you need to **specify an appropriate size** for the string variable. If the **size is too small**, then you will **not be able to store all the elements in the string**. On the other hand, if the **string size is large**, then **unnecessarily memory space will be wasted**.

4.2.2 Variable-length strings

- ✓ A better option is to use a variable length format in which the **string can be expanded or contracted to accommodate the elements in it**. For example, if you declare a string variable to store the name of a student. If a student has a long name of say 20 characters, then the string can be expanded to accommodate 20 characters. On the other hand, a student name has only 5 characters, then the string variable can be contracted to store only 5 characters. However, to use a variable-length string format you need a technique to indicate the end of elements that are a part of the string. This can be done either by using **length-controlled string or a delimiter**.

1. Length-controlled strings: In a length-controlled string, you need to specify the number of characters in the string.

2. Delimited strings: In this format, the string is ended with a delimiter such as comma, semicolon, colon, dash, null character etc. The delimiter is then used to identify the end of the string.



4.3 Operations on Strings

1. Finding Length of a String

- ✓ **The number of characters in a string constitutes the length of the string.** For example, `LENGTH("C PROGRAMMING IS FUN")` will return 20. Note that even blank spaces are counted as characters in the string.
- ✓ Figure 4.3 shows an algorithm that calculates the length of a string. In this algorithm, `I` is used as an index for traversing string `STR`. To traverse each and every character of `STR`, we increment the value of `I`. Once we encounter the null character, the control jumps out of the while loop and the length is initialized with the value of `I`.

```
Step 1: [INITIALIZE] SET I = 0
Step 2: Repeat Step 3 while STR[I] != NULL
Step 3:   SET I = I + 1
         [END OF LOOP]
Step 4: SET LENGTH = I
Step 5: END
```

Figure 4.3 Algorithm to calculate the length of a string

2. Converting Characters of a String into Upper Case

- ✓ We have already discussed that in the memory ASCII codes are stored instead of the real values. The **ASCII code for A–Z varies from 65 to 91 and the ASCII code for a–z ranges from 97 to 123**. So, if we have to convert a **lower case character into uppercase**, we just need to **subtract 32 from the ASCII value of the character**.
- ✓ Figure 4.4 shows an algorithm that converts the lower case characters of a string into upper case. In the algorithm, we initialize I to zero. Using I as the index of STR, we traverse each character of STR from Step 2 to 3. If the character is in lower case, then it is converted into upper case by subtracting 32 from its ASCII value. But if the character is already in upper case, then it is copied into the UPPERSTR string. Finally, when all the characters have been traversed, a null character is appended to UPPERSTR (as done in Step 4).

```
Step 1: [INITIALIZE] SET I=0
Step 2: Repeat Step 3 while STR[I] != NULL
Step 3: IF STR[I] >= 'a' AND STR[I] <= 'z'
        SET UPPERSTR[I] = STR[I] - 32
      ELSE
        SET UPPERSTR[I] = STR[I]
      [END OF IF]
      SET I = I + 1
    [END OF LOOP]
Step 4: SET UPPERSTR[I] = NULL
Step 5: EXIT
```

Figure 4.4 Algorithm to convert characters of a string into upper case

3. Converting Characters of a String into Lower Case

- ✓ If we have to convert an **upper case character into lower case**, we need to **add 32 to the ASCII value of the character**.
- ✓ Figure 13.8 shows an algorithm that converts the upper case characters of a string into lower case.
- ✓ In the algorithm, we initialize I to zero. Using I as the index of STR, we traverse each character of STR from Step 2 to 3. If the character is in upper case, then it is converted into lower case by adding 32 to its ASCII value. But if the character is already in lower case, then it is copied into the Lowerstr string. Finally, when all the characters have been traversed, a null character is appended to Lowerstr (as done in Step 4).

```
Step 1: [INITIALIZE] SET I = 0
Step 2: Repeat Step 3 while STR[I] != NULL
Step 3: IF STR[I] >= 'A' AND STR[I] <= 'Z'
        SET Lowerstr[I] = STR[I] + 32
      ELSE
        SET Lowerstr[I] = STR[I]
      [END OF IF]
      SET I = I + 1
    [END OF LOOP]
Step 4: SET Lowerstr[I] = NULL
Step 5: EXIT
```

Figure 13.8 Algorithm to convert characters of a string into lower case

4. Concatenating Two Strings to Form a New String

- ✓ If s1 and s2 are two strings, then concatenation operation produces a string which contains characters of s1 followed by the characters of s2.
- ✓ Figure 13.9 shows an algorithm that concatenates two strings. In this algorithm, we first initialize the two counters I and J to zero. To concatenate the strings, we have to copy the contents of the first string followed by the contents of the second string in the third string new_str. Steps 2 to 4 copies the

contents of first string in new_str. Likewise Steps 6 to 8 copies the contents of second string in new_str. After the contents have been copied, a null character is appended at the end of new_str.

```
Step 1: [INITIALIZE] I = 0 and J = 0
Step 2: Repeat Steps 3 to 4 while str1[i] != NULL
Step 3:     SET new_str[J] = str1[I]
Step 4:     Set I = I+1 and J = J+1
          [END of LOOP]
Step 5: SET I=0
Step 6: Repeat Steps 6 to 7 while str2[i] != NULL
Step 7:     SET new_str[J] = str2 [I]
Step 8:     Set I = I+1 and J = J+1
          [END of LOOP]
Step 9: SET new_str[J] = NULL
Step 10: EXIT
```

Figure 13.9 Algorithm to concatenate two strings

5. Appending a String to Another String

- ✓ **Appending one string to another string involves copying the contents of the source string at the end of the destination string.**
- ✓ For example, if S1 and S2 are two strings, then appending S1 to S2 means we have to add the contents of S1 to S2. So, S1 is the source string and S2 is the destination string. The appending operation would leave the source string S1 unchanged and the destination string S2 = S2 + S1.
- ✓ Figure 4.5 shows an algorithm that appends two strings. In this algorithm, we first traverse through the destination string to reach its end, i.e., reach the position where a null character is encountered. The characters of the source string are then copied into the destination string starting from that position. Finally, a null character is added to terminate the destination string

```
Step 1: [INITIALIZE] SET I=0 and J=0
Step 2: Repeat Step 3 while DEST_STR[I] != NULL
Step 3:     SET I = I + 1
          [END OF LOOP]
Step 4: Repeat Steps 5 to 7 while SOURCE_STR[J] != NULL
Step 5:     DEST_STR[I] = SOURCE_STR[J]
Step 6:     SET I = I + 1
Step 7:     SET J = J + 1
          [END OF LOOP]
Step 8: SET DEST_STR[I] = NULL
Step 9: EXIT
```

Figure 4.5 Algorithm to append a string to another string

6. Comparing Two Strings

- ✓ **If S1 and S2 are two strings, then comparing the two strings will give either of the following results:**
 - (a) S1 and S2 are equal
 - (b) S1>S2, when in dictionary order, S1 will come after S2
 - (c) S1 <S2, when in dictionary order, S1 precedes S2
- ✓ To compare the two strings, each and every character is compared from both the strings. If all the characters are the same, then the two strings are said to be equal.
- ✓ Figure 4.6 shows an algorithm that compares two strings.
- ✓ In this algorithm, we first check whether the two strings are of same length. If not, then there is no point in moving ahead as it straightaway means that the two strings are not same. However, if the two strings are of the same length, then we compare character by character to check if all the characters

are same. If yes, then variable same is set to 1 else if same = 0, then we check which string precedes the other in dictionary order and print the corresponding message.

```
Step 1: [INITIALIZE] SET I=0, SAME =0
Step 2: SET LEN1 = Length(STR1), LEN2 = Length(STR2)
Step 3: IF LEN1 != LEN2
        Write "Strings Are Not Equal"
    ELSE
        Repeat while I<LEN1
            IF STR1[I] == STR2[I]
                SET I = I + 1
            ELSE
                Go to Step 4
            [END OF IF]
        [END OF LOOP]
        IF I = LEN1
            SET SAME =1
            Write "Strings are Equal"
        [END OF IF]
Step 4: IF SAME = 0,
        IF STR1[I] > STR2[I]
            Write "String1 is greater than String2"
        ELSE IF STR1[I] < STR2[I]
            Write "String2 is greater than String1"
        [END OF IF]
    [END OF IF]
Step 5: EXIT
```

Figure 4.6 Algorithm to compare two strings

7. Reversing a String

- ✓ If S1="HELLO", then reverse of S1="OLLEH". **To reverse a string, we just need to swap the first character with the last, second character with the second last character, and so on.**
- ✓ Figure 4.7 shows an algorithm that reverses a string.
- ✓ In Step 1, I is initialized to zero and J is initialized to the length of the string-1. In Step 2, while loop is executed until all the characters of the string are accessed. In Step 3, we swap the ith character of STR with its jth character. In Step 4, the value of I is incremented and J is decremented to traverse STR in the forward and backward direction respectively.

```
Step 1: [INITIALIZE] SET I=0, J= Length(STR)-1
Step 2: Repeat Steps 3 and 4 while I < J
Step 3:     SWAP(STR(I), STR(J))
Step 4:     SET I = I + 1, J = J - 1
            [END OF LOOP]
Step 5: EXIT
```

Figure 4.7 Algorithm to reverse a string

8. Extracting a Substring from Left

- ✓ **In order to extract a substring from the main string we need to copy the content of the string starting from the first position to the nth position where n is the number of characters to be extracted.**
- ✓ For example, if S1= "Hello World", then Substr_Left(S1,7)=Hello w.
- ✓ The algorithm for extracting the first n characters from a string is shown below:
 - Step 1: [INITIALIZE] SET I = 0
 - Step 2: Repeat Step 3 to 4 while STR[I] != NULL AND I<N
 - Step 3: SET Substr[I] = STR[I]
 - Step 4: SET I = I+1

[END OF LOOP]

Step 5: SET Substr[I] = NULL

Step 6: EXIT

Figure 13.13 Algorithm to extract first n characters from a string

- ✓ In Step 1, we initialize the index variable I with zero. In Step 2, a while loop is executed until all the characters of STR have been accessed and I is less than N. In Step 3, the Ith character of STR is copied in the Ith character of Substr. In Step 4, the value of I is incremented to access the next character in STR. In Step 5, Substr is appended with a null character.

9. Extracting a Substring from Right

- ✓ **In order to extract a substring from the right side of the main string we need to first calculate the position from the left.**
- ✓ For example, if S1= "Hello World" and we have to copy 7 characters starting from the right then we have to actually start extracting characters from the 4th position. This is calculated by total number of characters - n.
- ✓ For example, if S1= "Hello World", then Substr_Right(S1, 7) = o World
- ✓ Figure 13.14 shows an algorithm that extracts n characters from the right of a string.

Step 1: [INITIALIZE] SET I = 0, J = Length(STR)-N

Step 2: Repeat Step 3 to 4 while STR[J] != NULL

Step 3: SET Substr[I] = STR[J]

Step 4: SET I = I+1, J=J+1

[END OF LOOP]

Step 5: SET Substr[I] = NULL

Step 6: EXIT

Figure 13.14 Algorithm to extract n characters from the right of a string

- ✓ In Step 1, we initialize the index variable I to zero and J to Length (STR)-N so that J points to the character from which the string has to be copied in the substring. In Step 2, a while loop is executed until the null character in STR is accessed. In Step 3, the Jth character of STR is copied in the Ith character of Substr. In Step 4, the value of I and J are incremented. In Step 5, Substr is appended with a null character.

10. Extracting a Substring from the Middle of a string

- ✓ **To extract a substring from a given string requires information about three things. The main string, the position of the first character of the substring in the given string and the number of characters/length of the substring.**
- ✓ For example, if we have a string,
str[] = "Welcome to the world of programming"
then
SUBSTRING (str, 15, 5) = World
- ✓ Figure 4.8 shows an algorithm that extracts the substring from a middle of a string.

```
Step 1: [INITIALIZE] Set I=M, J=0
Step 2: Repeat Steps 3 to 6
        while STR[I] != NULL and N>0
Step 3: SET SUBSTR[J] = STR[I]
Step 4: SET I = I + 1
Step 5: SET J = J + 1
Step 6: SET N = N - 1
        [END OF LOOP]
Step 7: SET SUBSTR[J] = NULL
Step 8: EXIT
```

Figure 4.8 Algorithm to extract a substring from the middle of a string

- ✓ In this algorithm, we initialize a loop counter I to M. i.e., the position from which the characters have to be copied. Steps 3 to 6 are repeated until N characters have been copied. With every character copied, we decrement the value of N. The characters of the string are copied into a string called substr. At the end a null character is appended to substr to terminate the string.

11. Inserting a String in Another String

- ✓ **The insertion operation inserts a string S in the main text T at the kth position. The general syntax of this operation is INSERT(text, position, string).**
- ✓ For example, INSERT("XYZXYZ",3, "AAA") = "XYZAAAXYZ" Figure 4.9 shows an algorithm to insert a string in a given text at the specified position. This algorithm first initializes the indices into the string to zero. From Steps 3 to 5, the contents of NEW_STR are built. If I is exactly equal to the position at which the substring has to be inserted, then the inner loop copies the contents of the substring into NEW_STR. Otherwise, the contents of the text are copied into it.

12. Indexing

- ✓ **This operation returns the position in the string where the string pattern first occurs.**
- ✓ For example, INDEX("Welcome to the world of programming", "world") = 15
- ✓ However, if the pattern does not exist in the string, the INDEX function returns 0.

13. Deleting a string from the Main String

- ✓ **The deletion operation deletes a substring from a given text.**
- ✓ We can write it as DELETE(text, position, length).
- ✓ For example, DELETE("ABCDXXXABCD", 4, 3) = "ABCDABCD"

14. Replacing a Pattern with Another Pattern in a String

- ✓ **The replacement operation is used to replace the pattern P1 by another pattern P2.**
- ✓ This is done by writing REPLACE(text, pattern1 , pattern2).
- ✓ For example, ("AAABBBCCCC", "BBB", "X") = AAAXCCC
("AAABBBCCCC", "X", "YYY")= AAABBBCC

4.3 Miscellaneous String and Character Functions

4.3.1 Character Manipulation Functions

- ✓ Table 8.1 illustrates some character functions contained in ctype.h.

Prototype	Function description
<code>int isblank(int c);</code>	Returns a true value if c is a <i>blank character</i> that separates words in a line of text and 0 (false) otherwise. [Note: This function is not available in Microsoft Visual C++.]
<code>int isdigit(int c);</code>	Returns a true value if c is a <i>digit</i> and 0 (false) otherwise.
<code>int isalpha(int c);</code>	Returns a true value if c is a <i>letter</i> and 0 otherwise.
<code>int isalnum(int c);</code>	Returns a true value if c is a <i>digit</i> or a <i>letter</i> and 0 otherwise.
<code>int isxdigit(int c);</code>	Returns a true value if c is a <i>hexadecimal digit character</i> and 0 otherwise. (See Appendix C for a detailed explanation of binary numbers, octal numbers, decimal numbers and hexadecimal numbers.)
<code>int islower(int c);</code>	Returns a true value if c is a <i>lowercase letter</i> and 0 otherwise.
<code>int isupper(int c);</code>	Returns a true value if c is an <i>uppercase letter</i> and 0 otherwise.
<code>int tolower(int c);</code>	If c is an <i>uppercase letter</i> , tolower returns c as a <i>lowercase letter</i> . Otherwise, tolower returns the argument unchanged.

<code>int toupper(int c);</code>	If <i>c</i> is a <i>lowercase letter</i> , <code>toupper</code> returns <i>c</i> as an <i>uppercase letter</i> . Otherwise, <code>toupper</code> returns the argument unchanged.
<code>int isspace(int c);</code>	Returns a true value if <i>c</i> is a <i>whitespace character</i> —newline (<code>'\n'</code>), space (<code>' '</code>), form feed (<code>'\f'</code>), carriage return (<code>'\r'</code>), horizontal tab (<code>'\t'</code>) or vertical tab (<code>'\v'</code>)—and 0 otherwise.
<code>int iscntrl(int c);</code>	Returns a true value if <i>c</i> is a <i>control character</i> and 0 otherwise.
<code>int ispunct(int c);</code>	Returns a true value if <i>c</i> is a <i>printing character other than a space, a digit, or a letter</i> and returns 0 otherwise.
<code>int isprint(int c);</code>	Returns a true value if <i>c</i> is a <i>printing character including a space</i> and returns 0 otherwise.
<code>int isgraph(int c);</code>	Returns a true value if <i>c</i> is a <i>printing character other than a space</i> and returns 0 otherwise.

Fig. 8.1 | Character-handling library (`<ctype.h>`) functions. (Part 2 of 2.)

4.3.2 String Manipulation Functions

✓ The standard library 'string.h' contains many functions for the string manipulation.

1. `strlen(str)`: The length of a string

- ✓ The 'strlen()' function can be used to find the length of the string in bytes.
- ✓ This function calculates the length of the string excluding the 'null character'. i.e., the function returns the number of characters in the string.

Syntax: `size_t strlen(const char *str);`

Ex: Write a C program to demonstrate the usage of strlen().

```
#include<stdio.h>
#include<string.h>
void main()
{
    char str[] = "HELLO WORLD!";
    printf("\n The length of the string is: %d", strlen(str));
}
```

Output:

The length of the string is: 5

2. `strcpy ( )`: String Copy

- ✓ The 'strcpy()' function copies the contents of source string `str2` to destination string `str1` including `'\0'`.
- ✓ The `strcpy()` function copies characters from the source string to the destination string until it finds null character. It returns the argument `str1`.

Syntax: `char * strcpy(char *str1, const char *str2);`

Where,

`str1`: it is the destination string

`str2`: it is the source string.

Ex: Write a C program to demonstrate the usage of strcpy().

```
#include<stdio.h>
#include<string.h>
```

```
void main()
```

```

{
    char str1[10], str2[10]= "JAIN";
    strcpy(str1,str2);
    printf("The Source String=%s\n The Destination String=%s", str1,str2);
}

```

Output:

The Source String= JAIN

The Destination String= JAIN

3. strncpy(str1,str2,n): String Number Copy

- ✓ 'strncpy()' function is used to copy 'n' characters from the source string str2 to the destination string str1.

Syntax : `char *strncpy(char *str1, const char *str2, size_t n);`

Where,

str1: it is the destination string.

str2: it is the source string.

n: n is the number of characters to be copied into destination string.

Ex: Write a C program to demonstrate the usage of strncpy().

```

#include<stdio.h>
#include<string.h>
void main()
{
    char str1[10], str2[10]= "Computer";
    strncpy(str1,str2,3);
    printf ("The Source String=%s\n The Destination String=%s", str1,str2);
}

```

Output:

The Source String= Computer

The Destination String= Com

4. strcat(str1,str2): String Concatenate(Joining two strings together)

- ✓ The strcat function appends the string pointed to by str2 to the end of the string pointed to by str1.
- ✓ The 'strcat()' function is used to concatenate or join the two strings.
- ✓ The 'strcat()' function copies all the characters of string str2 to the end of string str1.
- ✓ The NULL character of string str1 is replaced by the first character of str2.

Syntax: `char *strcat(char *str1, const char *str2);`

Where,

str1: It is the first string

str2: It is the second string

Ex: Write a C program to demonstrate the usage of strcat().

```

#include<stdio.h>
#include<string.h>
void main()
{
    char str1[15]= "Good";
    char str2[15]= "Morning";
    strcat(str1,str2);
    printf("The concatenated String=%s",str1);
}

```

Output:

The concatenated String=GoodMorning.

5. strncat(str1,str2,n)- String Number Concatenate

- ✓ The 'strncat()' function is used to concatenate or join n characters of string str2 to the end of string str1.

Syntax: `char *strcat(char *str1, const char *str2, size_t n);`

Where,

str1: It is the first string

str2: It is the second string

n: It is the number of characters of string s2 to be concatenated.

Ex: Write a C program to demonstrate the usage of strncat().

```
#include<stdio.h>
#include<string.h>
void main()
{
    char str1[15]= "Good";
    char str2[15]= "Morning";
    strncat(str1,str2,4);
    printf("The concatenated String=%s",str1);
}
```

Output:

The concatenated String=GoodMorn.

6. strcmp(str1,str2): String Compare

- ✓ This function is used to compare two strings.
- ✓ The comparison starts with first character of each string. The comparison continues till the corresponding characters differ or until the end of the character is reached.
- ✓ The following values are returned after comparison:
 1. If two strings are equal, the function returns 0.
 2. If str1 is greater than str2, a positive value is returned.
 3. If str1 is less than str2, then the function returns a negative value.

Syntax: `int strcmp(const char *str1, const char *str2);`

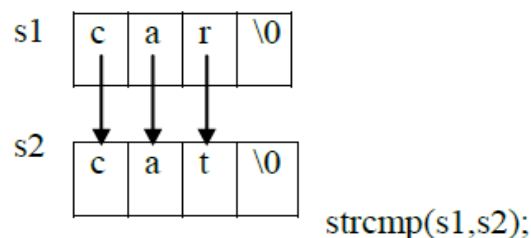
Where,

str1: It is the first string.

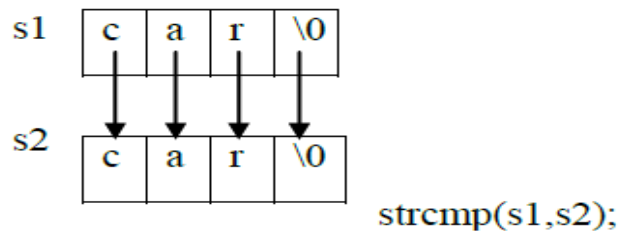
str2: It is the second string.

Ex:

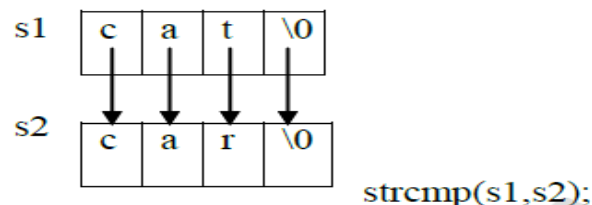
- ✓ "car" and "cat" are different strings. The characters 'r' and 't' have different ASCII values. It returns negative value since ASCII value of 'r' is less than the ASCII value of 't'.



- ✓ “car” and “car” are same, the function returns 0.



- ✓ “cat” and “car” are different strings. The characters ‘t’ and ‘r’ have different ASCII values. It returns positive value since ASCII value of ‘t’ is greater than the ASCII value of ‘r’.



Ex: Write a C program to demonstrate the usage of strcmp().

```
#include<stdio.h>
#include<string.h>
void main()
{
    char str1[10]="Hello";
    char str2[10]="Hey";
    if(strcmp(str1,str2)==0)
        printf("The two strings are identical");
    else
        printf("The two strings are not identical");
}
```

Output:

The two strings are not identical

7. strncmp(str1,str2,n): String Number Compare

- ✓ This function is used to compare first n number of characters in two strings.
- ✓ The comparison starts with first character of each string. The comparison continues till the corresponding characters differ or until the end of the character is reached or specified numbers of characters have been tested.
- ✓ The following values are returned after comparison:
 1. If two strings are equal, the function returns 0.
 2. If str1 is greater than str2, a positive value is returned.
 3. If str1 is less than str2, then the function returns a negative value.

Syntax: int strcmp(const char *str1, const char *str2, size_t n);

Where,

str1: It is the first string.

str2: It is the second string.

n: It is the number of characters to be compared.

Ex: Write a C program to demonstrate the usage of strcmp().

```
#include<stdio.h>
```

```
#include<string.h>
void main()
{
    char str1[10]="Hello";
    char str2[10]="Hey";
    if(strncmp(str1,str2,2)==0)
        printf("The first two characters in the strings are identical");
    else
        printf("The first two characters in the strings are not identical");
}
```

Output:

The first two characters in the strings are identical

8. strchr()

- ✓ It takes a string and a character as input and finds out the first occurrence of the given character in the string pointed to by the argument str.
- ✓ It will return the pointer to the first occurrence of that character; if found otherwise, return Null.

Syntax:

char *strchr(const char *str, int c);

Ex: Write a C program to demonstrate the usage of strchr().

```
#include <stdio.h>
#include <string.h>
void main()
{
    char string1[30] = "I love to write.";
    char *pos;
    pos= strchr(string1, 'w');
    if(pos)
        printf("w is found at position %d", pos);
    else
        printf("w is not found");
}
```

Output:

w is found at position 10

9. strrchr()

- ✓ It takes a string and a character as input and finds out the last occurrence of a given character in that string.
- ✓ It will return the pointer to the last occurrence of that character if found otherwise, return Null.

Syntax:

char *strrchr(const char *str, int c);

Ex: Write a C program to demonstrate the usage of strrchr().

```
#include <stdio.h>
```

```
#include <string.h>
void main()
{
    char string1[30] = "Programming in C.";
    char *pos;
    pos= strrchr(string1, 'n');
    if(pos)
        printf("The last position of n is %d", pos);
    else
        printf("n is not found");
}
```

Output:

The last position of n is 13.

10. strspn()

✓ The function returns the index of the first character in str1 that doesn't match any character in str2.

Syntax:

size_t strspn(const char *str1, const char *str2);

Ex: Write a C program to demonstrate the usage of strspn().

```
#include <stdio.h>
#include <string.h>
main()
{
    char str1[] = "HAPPY BIRTHDAY TO YOU";
    char str2[]="HAPPY BIRTHDAY JOE";
    printf("The position of the first character in str2 that does not
        match with that in str1 is %d", strspn(str1,str2));
}
```

Output:

The position of the first character in str2 that does not match with that in str1 is 15

11. strcspn()

✓ The function returns the index of the first character in str1 that matches any of the characters in str2.

Syntax:

size_t strcspn(const char *str1, const char *str2);

Ex: Write a C program to demonstrate the usage of strcspn().

```
#include <stdio.h>
#include <string.h>
void main()
{
    char str1[] = "PROGRAMMING IN C";
    char str2[]="IN";
    printf("The position of the first character in str2 that match with
        that in str1 is %d", strcspn(str1,str2));
}
```

Output:

The position of the first character in str2 that match with that in str1 is 8

12. strpbrk()

- ✓ The function strpbrk() returns a pointer to the first occurrence in str1 of any character in str2, or NULL if none are present.
- ✓ The only difference between strpbrk() and strcspn() is that strcspn() returns the index of the character and strpbrk() returns a pointer to the first occurrence of a character in str2.

Syntax:

char *strpbrk(const char *str1, const char *str2);

Ex: Write a C program to demonstrate the usage of strpbrk().

```
#include <stdio.h>
#include <string.h>
void main()
{
    char str1[] = "PROGRAMMING IN C";
    char str2[]="AB";
    char *ptr = strpbrk(str1,str2);
    if(ptr== NULL)
        printf("\n No character matches in the two strings");
    else
        printf("\n character in str2 matches with that in str1");
}
```

Output:

character in str2 matches with that in str1

13. strtok()

- ✓ The strtok() function is used to isolate sequential tokens in a null –terminated string, str.
- ✓ It returns a pointer to the beginning of each subsequent token in the string, after replacing the token itself with a null character.
- ✓ When all tokens are left , a null pointer is returned.

Syntax:

char *strtok(char *str1, const char *delimiter);

Ex: Write a C program to demonstrate the usage of strtok().

```
#include <stdio.h>
#include <string.h>
void main()
{
    char str[] = "Hello, to, the, World of, Programming";
    char delim[] = ",";
    char *result;
    result = strtok(str,delim);
    while(result!= NULL)
    {
        printf("\n %s", result);
        result = strtok(NULL,delim);
    }
}
```

Output:

Hello
to
the
World of
Programming

4.3.3 String Manipulation functions present in <stdlib.h>

1. strtol()

- ✓ The strtol() function converts the string pointed by str to a long value.
- ✓ The function skips the leading white space characters and stops when it encounters the first non-numeric character.
- ✓ It stores the address of the first invalid character in str in *end.
- ✓ You may pass NULL instead of *end if you do not want to store any invalid characters anywhere.
- ✓ Third argument base specifies whether the number is Hexadecimal, binary, octal or decimal representation.

Syntax:

long strtol(const char *str, char **end, int base);

Ex: Write a C program to demonstrate the usage of strtol().

```
#include <stdio.h>
#include <stdlib.h>
void main()
{
    long num;
    num = strtol("12345 Decimal Value", NULL, 10);
    printf("%ld", num);
    num = strtol("65432 Octal Value", NULL, 8);
    printf("%ld", num);
    num = strtol("10110101 Binary Value", NULL, 2);
    printf("%ld", num);
    num = strtol("A7CB4 Hexadecimal Value", NULL, 16);
    printf("%ld", num);
}
```

Output:

12345
27418
181
687284

2. strtod()

- ✓ The function accepts a string str that has an optional plus(‘+’) or minus sign(‘-’) followed by either:
- ✓ A decimal number containing a sequence of decimal digits optionally consisting of a decimal point, or
- ✓ A Hexadecimal number consisting of a “OX” or “Ox” followed by a sequence of hexadecimal digits optionally containing a decimal point.
- ✓ In both the cases the number should be optionally followed by an exponent (‘E’ or ‘e’ for decimal constants and ‘P’ or ‘p’ for hexadecimal constants).

Syntax:

double strtod(const char *str, char **end);

Ex: Write a C program to demonstrate the usage of strtod().

```
#include<stdio.h>
#include<stdlib.h>
void main()
{
    double sum;
    num = strtod("123.345abcdefg", NULL);
    printf("%lf", num);
}
```

Output:

123.345000

3. atoi()

- ✓ The atoi() function converts a given string passed to it as an argument into an integer.
- ✓ The function returns that integer to the calling function.
- ✓ However, the string should start with a number.
- ✓ The function will stop reading from the string as soon as it encounters a non-numerical character.

Syntax:

int atoi(const char *str);

Example:

```
i = atoi("123.456");
Result: i=123.
```

4. atof()

- ✓ This function converts the string that it accepts as an argument into a double value and then returns that to the calling function.
- ✓ The string can be terminated with any non-numerical character other than "E" or "e".

Syntax:

double atof(const char *str);

Example:

```
x = atof("12.39 is the answer");
Result: x=12.39
```

5. atol()

- ✓ This function converts the string into a long int value.
- ✓ This function will read from the string until it finds any character that should not be in long.

Syntax:

long atol(const char *str);

Example:

```
x= atol("12345.6789");
Result: x = 12345L.
```

4.4 Arrays of Strings

- ✓ An array of strings is declared as

<data_type> <array_name> [row_size][column_size];

Here, the first index row_size will specify how many strings are needed and the second index column_size will specify the length of every individual string.

Ex: char names[20][30];

- ✓ So here, we will allocate space for 20 names where each name can be a maximum 30 characters long.
- ✓ Let us see the memory representation of an array of strings. If we have an array declared as char name[5][10] = {"Ram", "Mohan", "Shyam", "Hari", "Gopal"};

Then in the memory, the array will be stored as shown in Fig. 4.13.

name[0]	R	A	M	'\0'					
name[1]	M	O	H	A	N	'\0'			
name[2]	S	H	Y	A	M	'\0'			
name[3]	H	A	R	I	'\0'				
name[4]	G	O	P	A	L	'\0'			

Figure 4.13 Memory representation of a 2D character array

- ✓ By declaring the array names, we allocate 50 bytes. But the actual memory occupied is 27 bytes. Thus, we see that about half of the memory allocated is wasted.
- ✓ Figure 4.14 shows an algorithm to process individual string from an array of strings. In Step 1, we initialize the index variable I to zero. In Step 2, a while loop is executed until all the strings in the array are accessed. In Step 3, each individual string is processed.

```
Step 1: [INITIALIZE] SET I=0
Step 2: Repeat Step 3 while I < N
Step 3:   Apply Process to NAMES[I]
         [END OF LOOP]
Step 4: EXIT
```

Figure 4.14 Algorithm to process individual string from an array of strings

Ex: Write a C Program to Read and Print the Names of N Students of a Class.

```
#include<stdio.h>
void main()
{
    char names[5][10];
    int i, n;
    printf("\n Enter the number of students : ");
    scanf("%d", &n);
    printf("\n Enter the names of students :");
    for(i=0;i<n;i++)
    {
        scanf("%s",names[i]);
    }
    printf("\n Names of the students are : \n");
    for(i=0;i<n;i++)
        puts(names[i]);
}
```